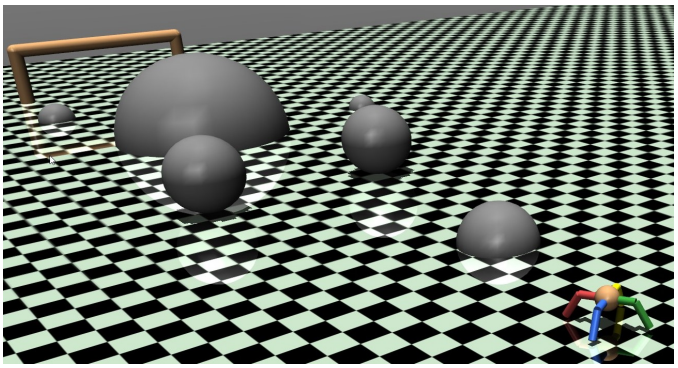
Convex Heuristics for Limb Placement and Navigation

1st Polo Contreras
Dept. of Electrical Engineering
Stanford University
jcontr83@stanford.edu

2nd Preston Rogers
Dept. of Electrical Engineering
Stanford University
parogers@stanford.edu

I. Introduction



One of the contemporary challenges in robotic locomotion is the efficient calculation of the movements necessary both for limb collocation and navigation, such that the agent can react to its environment and effectively maneuver to a target location with minimal or no guidance from a human operator. This project casts this as two optimization problems: the problem of tracing a path between two locations while dodging an obstacle is cast as a cost minimization problem with convex approximations of nonconvex constraints, while the problem of calculating the movements necessary to move a limb to a certain position and take a step is also structured as a cost minimization problem with quasilinearized dynamics. The combination of these two approaches allows the agent to use principles of cost minimization to model its environment and thus react to its surroundings, using only details of its goal and environmental geometry to inform its decisions.

II. Related Works

There has been extensive research in the area of using convex optimization heuristics for navigation and limb collocation; as these are nonconvex problems, a degree of adaptation must be performed to effectively balance accuracy and efficiency. Previous implementations, such as those of Valenzuela [1], formulate the calculation of step placements for a complete trajectory before any action is performed. This particular implementation formulates the generation of a minimum cost trajectory as a guiding principle but limb selection and step placement decisions are made in real time, with the use of memoization to refer to previous solutions to reduce overall computational time.

For end-effector placement, Sequential Convex Programming (SCP) has permeated the literature due to the nonlinear dynamics involved. In one notable example, Saramago and Junior used sequential minimization to calculate a robot manipulator trajectory that seeks to avoid obstacles [2]. The objective uses regularization terms that establishes varying importance in travel-time, sum of quadratic forces applied, and the penalty parameter to guarantee free-collision motion. Similar to this implementation, our end-effector trajectory objective minimizes the sum of quadratic torque applied. We do not include a regularized term in the objective for our path calculation; rather, we opt to make obstacle avoidance a hard constraint.

The linearization of constraints for the purpose of convexifying non-convex constraints is also a well-explored topic. For 3-Dimensional trajectory planning for aircrafts, it is essential to consider the impact of No-Fly Zone (NFZ). Xinfu Liu et al. establish that these constraints, which can be viewed as distances from cylinders, are concave functions that must be convexified [4]. They propose a first-order Taylor approximation that is updated after each convex sub-problem solve. A similar approach is used to achieve locally optimal solutions for this implementation.

III. Dataset

The dataset taken in by the agent consists specifically of its own world space position and rotation, the position and rotation of its limbs relative in the robot control space, the masses and dimension of its body and limbs, as well as the location of the end goal and location and dimensions of the obstacles in the world space. This data is provided unaltered to the agent at each time step and is not impacted by simulated sensor measurement error.

IV. Baseline

There is, unfortunately, not a straightforward oracle that could be used to evaluate performance for this project; in effect, calculation of the optimal solution for a given configuration of the terrain, the location of the agent and the location of the goal might involve the calculation of all possible combinations of joint configurations and robot positions in

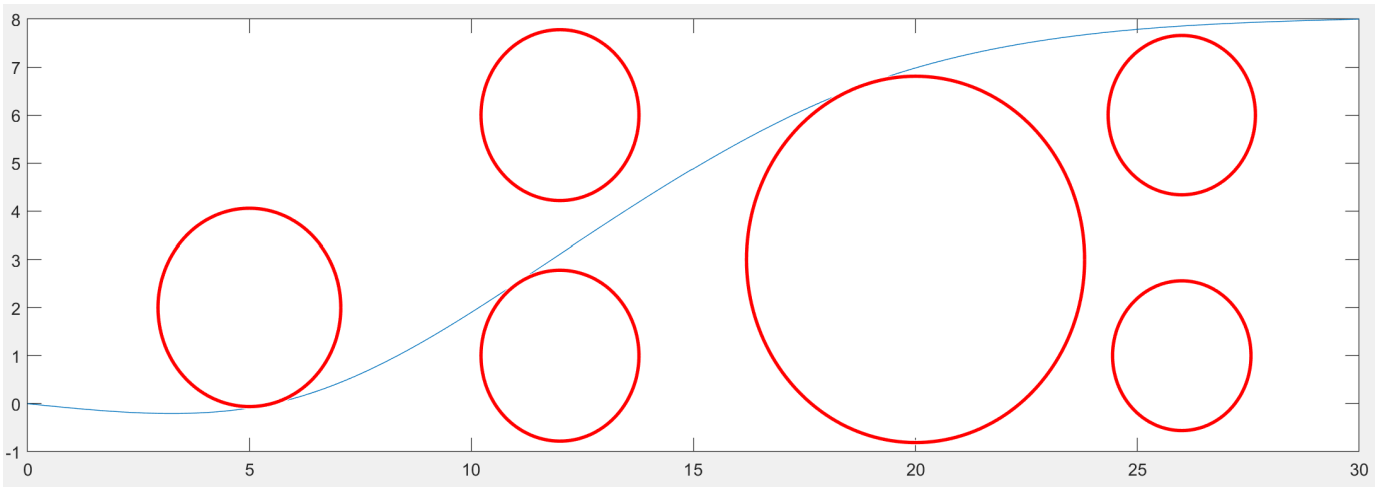


Fig. 1. Calculation of minimum cost trajectory while avoiding obstacles. For navigation purposes, obstacles use both the physical dimensions of the obstacle and the dimensions of the agent, and the trajectory guides the position of the center of the agent.

the simulated physical space; this is an intractable problem. The baseline performance metric then is simply the minimum cost path computed between the agent and the goal, discussed in more detail in the section below. Accordingly, this path is target path by which we can benchmark performance of the true agent path result from the limb placement solver.

V. Main Approach

In this approach, the agent first uses its knowledge of the environment to determine a minimum cost path to a specified goal location. Knowing its starting location, the location of the goal, and the location and size of each of the obstacles in the space, the algorithm uses the following objective function to determine a minimum cost trajectory between the two positions while remaining a given distance away from specific obstacle locations:

$$
\begin{aligned}
& p_j \equiv \begin{bmatrix} p_{1j} \\ p_{2j} \end{bmatrix} \\
& \underset{p}{\text{minimize}} \quad \sum_{j=2}^{N-1} \|p_{j-1} - 2p_j + p_{j+1}\|_2^2 \\
& \text{subject to} \quad p_1 = p_{initial} \\
& \qquad\qquad p_N = p_{final} \\
& \qquad\qquad \|p_j - p_o\|_2 \geq r_o \; \forall \; o = 1, \ldots, O
\end{aligned}
$$

Here, j indicates each step and N is the total number of steps. Each position p_j is a vector containing the position at a given step; the initial and final positions in the path are set to be the initial position of the agent and the location of the goal. The O constraints where $\|p_j - p_o\|_2 \geq r_o$ (where p_o is the position of an obstacle and r_o is the minimal Euclidean distance that the center of the agent can be from the position of the obstacle while avoiding a collision) are not convex, and a convex approximation must be used.

To elaborate, the minimization problem is modified to only include convex constraints that are approximations of the true constraints where the true feasible region is a superset of the intersection of convex constraints. The convex version of the problematic, non-convex, constraint ($\|p_j - R_o\|_2 \geq r_o \; \forall o = 1, \ldots, O$) is a standard first-order Taylor approximation based on the current iteration's path, with k indicating the number of an iteration on the solution:

$$
\begin{aligned}
& \|p_j^{(k)} - p_o\|_2 + ((p_j^{(k)} - p_o)/\|p_j^{(k)} - p_o\|_2)^T (p_j - p_j^{(k)}) \geq r_o \\
& \implies \|p_j - p_o\|_2 \geq r_o
\end{aligned}
$$

Note: In this formulation, we use the fact that we do not need to include information regarding the non-differential point that occurs when ($p_j^{(k)} = p_o$) because the probability of such an occurrence is nearly zero for $k = 1$ and for $k > 1$, the trajectory points are repelled from said non-differential points.

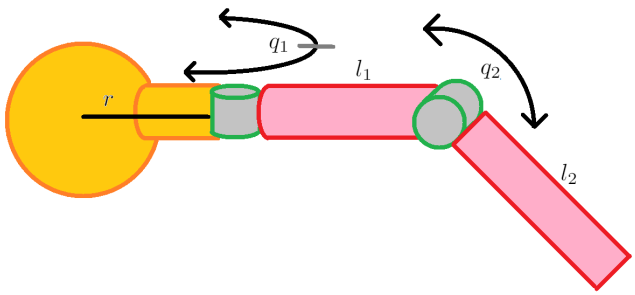


Fig. 2. Diagram of a single arm on the ant model.

Several different methods can be used to determine the number of iterations to run this procedure; this particular implementation checks for the convergence of the objective function value and considers the process complete when the cost associated with the latest iteration and its preceding iteration are sufficiently close (measured by the absolute value

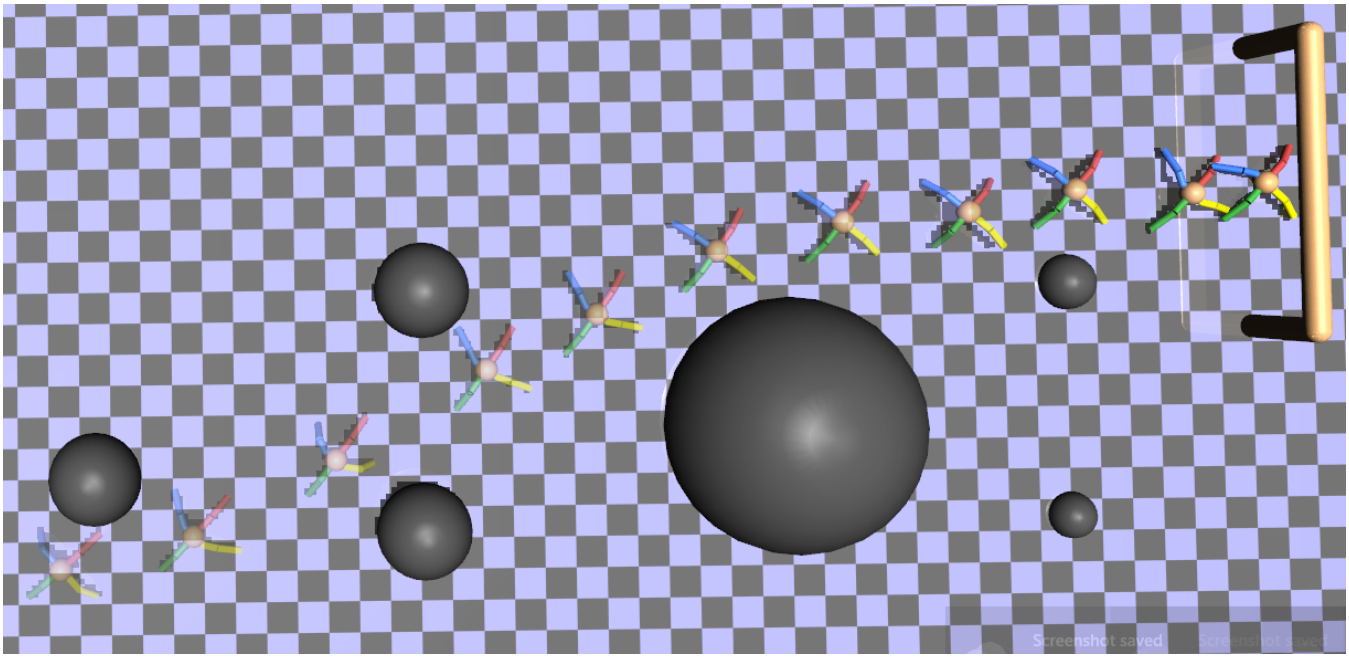


Fig. 3. Renders of agent movements and position over time, in simulated environment.

of the difference between these costs). Once this process is complete, the path corresponding to the latest iteration is saved and used thereafter. It is stored as a list of locations in two dimensions, corresponding to locations on the surface of the virtual environment.

Once the path has been selected and stored, the agent compares its current position with the positions detailed in the path and selects one of these positions as its next target. Using the position of this target location relative to its own location, the agent selects a combination of limbs to use in order to move towards the target location, as well as the final positions to which these limbs must move, and the process of determining the intervening actuator movements for each limb is cast as a cost minimization problem.

The cost associated with the specific movements of the actuators is described as being proportional to the torque required to move the limbs, the implication being that selecting the minimum torque required would allow the actuators move while expending as little energy as possible. Due to the inverse dynamics of limb movements being non-affine, it is necessary to approximate the dynamics to utilize convex heuristics. The approximation selected for this implementation was quasilinearization due to its low computational cost in regards to the pre-computation required for a sub-problem solve. This choice characterizes the determination of the minimum cost trajectory as a series of convex approximations of the original problem. At each iteration, a trust-region is modified to reflect the accuracy of the quasilinearized dynamics used for the iteration's optimizing q^*:

$$
\begin{aligned}
& q_i \equiv \begin{bmatrix} q_{1i} \\ q_{2i} \end{bmatrix} \\
& \underset{q}{\text{minimize}} \quad \sum_{i=1}^{n} \|\tau\|_2^2 \\
& \text{subject to} \quad M(q_i^{(k)}) \frac{q_{i+1} - 2q_i + q_{i-1}}{h^2} \\
& \qquad + W\left(q_i^{(k)}, \frac{q_{i+1}^{(k)} - q_{i-1}^{(k)}}{2h}\right) \frac{q_{i+1} - q_{i-1}}{2h} \\
& \qquad = \tau_i, \quad i = 1, \ldots, n-1
\end{aligned}
$$

Here, q_i describes the angles of each of the joints in limb that is being moved in a given time step. M is the quasilinearized version of the mass matrix and W is the quasilinearized version of the Coriolis matrix, each used to represent the dynamics of the movements of the limb. The amount of time elapsed between inputs by our controller is h, while the vector of torques applied on the joints at a given time step is τ_i. Once the minimum cost path has been determined, it is saved in case the same step maneuver is required again for the same limb. This allows the agent to look up previous solutions and avoid the computationally intensive task of computing the same movement multiple times. After the motions for the next step are computed or retrieved, the action is executed and the agent compares its current position to the positions in the path, updating them if necessary and selecting the next limbs to use along with their required final positions.

As the agent approaches a given target location, it will gradually update its target location to further locations in the path and thus follow the path until it selects the goal location.

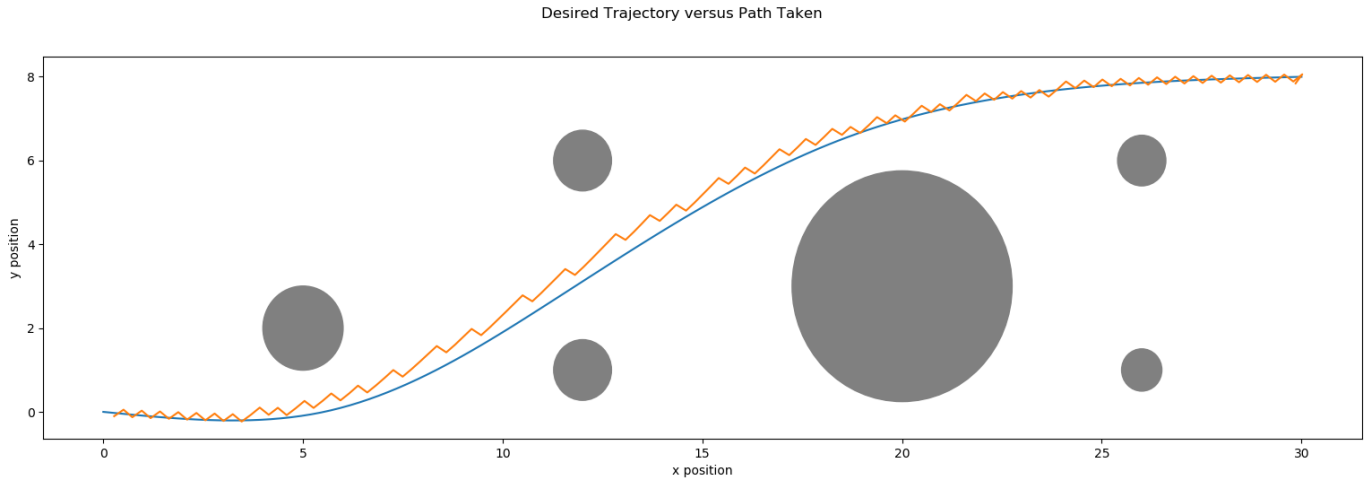


Fig. 4. Desired trajectory determined by navigation algorithm, path followed by agent in simulated environment, and positions of obstacles in simulated environment.

It will then move towards the goal position until it is within a specified horizontal distance of the position, after which the algorithm will terminate.

The agent and environment are simulated using MuJoCo (Multi-Joint dynamics with Contact), a popular physics engine for robotics research [2]. Using a modified version of the "Ant" environment from OpenAI Gym, a four-limbed agent with four identical limbs with two joints each, the agent itself is simulated together with the obstacles and environment while the movements of the agent are implemented at the level of the individual joints.

As MuJoCo does not provide an extensive commercial-grade convex optimization library [6], this project uses an interface with Python to calculate solutions to convex optimization problems (via CVXPY, a Python-embedded modeling language for convex problems) and issue instructions to the agent in MuJoCo.

VI. Evaluation Metric

The evaluation metric for success is built into the Python interface: after the agent begins to move, the simulation will continue until the agent comes within a specified distance of the goal location. Success is measured by the capability of the agent to autonomously reach the goal location, without colliding with obstacles in the environment, and to do so reliably (the MuJoCo environment provides a degree of randomization of the position of the agent at the beginning of the simulation).

VII. Results and Analysis

In this section, we will present a selection of results from our experiments and give an explanation for these results.

After being placed in the environment, the agent calculates the trajectory to the end location and stores it, approximately following it until it reaches the goal location. Multiple initializations, where randomization in MuJoCo changes the angle of the agent with respect to its surroundings at the beginning of its movements, do not affect the generation of a path to

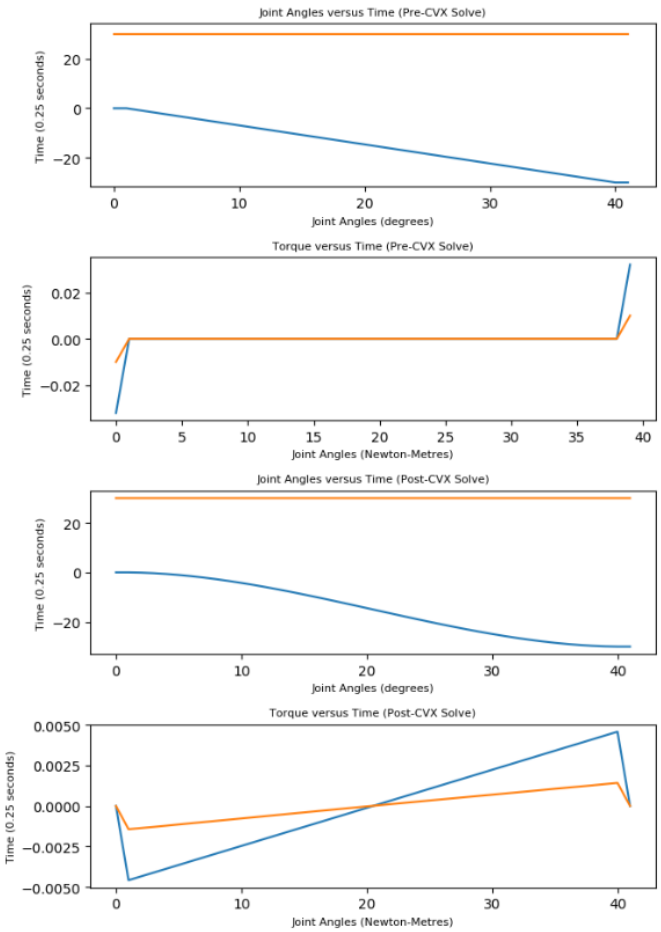


Fig. 5. Comparison of limb angles and torques using linear interpolation and solution of quasilinearized problem.

the goal location. Although this randomization can affect the specific movements performed by the agent in order to follow this trajectory, the agent is still able to reliably reach the goal without colliding with obstacles.

Reviewing the motions calculated by the convex solver and the calculated torques show a marked reduction of the objective value as the limb moves from its default position to take a step forward: specifically, the total value of the objective function (the sum of the squares of torques) using the solution of the quasilinearized problem reduces the value of the objective function by approximately 86.75% when compared to the value of the objective function corresponding to linear interpolation between the starting and ending positions when one leg is moved forward. Using two legs at a time, the agent moves towards its current target location using the two legs that will allow it to make the most progress in that direction and updates its target location in real time, only stopping on a single location once it has reached the goal and completed the overall navigation task.

VIII. Future Work

While the agent has demonstrated a capacity to effectively navigate the simulated terrain, there are several avenues for further development of the techniques used, both to make these methods more efficient and to generalize them to more environments and robot designs. The use of the convex solver CVXPY showcases a potential area for improvement: although the current implementation works well in calculating the minimum cost trajectory for a specified limb movement, speeding the process of calculating this trajectory could allow an agent to respond in real time to a changing environment while dynamically selecting the next positions for its limbs. In the current implementation there is a significant difference in the time taken between calculating a new limb movement and reading a previously stored solution, and while the inherent complexity of the problem makes such a discrepancy inevitable the agent may be able to respond more efficiently to unfamiliar situations if this discrepancy is reduced. Iterating on previous solutions, with the number of iterations used in calculating a new solution being specifically based on the divergence of the new solution from the previous solution, is one potential avenue for exploration.

Future work can also be performed on allowing the trajectory generated by the navigation algorithm to be edited in real time. Currently, convex approximations are used for the nonconvex constraints to ensure that the agent does not collide with stationary obstacles, while the step where the path is generated and the step where the path is used are wholly separate. Although effective in this context, this relies on the positions of potential obstacles being known ahead of time while the trajectory is being calculated. Including the ability to edit the trajectory during movement would allow an agent to use the same convex heuristics to adapt the path taken in response to changes in its knowledge of the environment, allowing it to use incomplete information of its surroundings (in a knowledge base that is amended as the agent progresses) or to react in real time to changing conditions in the environment.

Most importantly, the general principle of using the navigation algorithm to trace a path in the environment and using limb collocation to follow this path to a goal location can be expanded to a variety of robot designs. The current implementation is highly specialized for the geometry of the agent, and the ability to expand this to a number of different designs and means of locomotion allows us not only to expand the number of designs that we can use but also generalize the principle to other forms of locomotion, such as autonomous aircraft and watercraft, and the types of terrain that we are able to traverse with both legged agents and other mobile robot designs.

IX. Code

All code used in this project can be found in the github repository at https://github.com/PoloContreras/LegPlanningProject. See the README in the repository for details on the software package requirements and commands for how to run the simulation.

References

[1] A.K. Valenzuela, "Multiscale 3D Navigation," Doctoral Dissertation, Massachusetts Institute of Technology 2016.

[2] S. Saramago and V. Junior, "Optimal trajectory planning of robot manipulators in the presence of moving obstacles", Mechanism and Machine Theory, vol. 35, no. 8, pp. 1079-1094, 2000. Available: 10.1016/s0094-114x(99)00062-2.

[3] X. Liu, Z. Shen and P. Lu, "Entry Trajectory Optimization by Second-Order Cone Programming", Journal of Guidance, Control, and Dynamics, vol. 39, no. 2, pp. 227-241, 2016. Available: 10.2514/1.g001210.

[4] I. Mordatch, Papers.nips.cc, 2020. [Online]. Available: http://papers.nips.cc/paper/5764-interactive-control-of-diverse-complex-characters-with-neural-networks.pdf. [Accessed: 20- May- 2020].

[5] E. Otten, "Inverse and forward dynamics: models of multi–body systems", Philosophical Transactions of the Royal Society of London. Series B: Biological Sciences, vol. 358, no. 1437, pp. 1493-1500, 2003. Available: 10.1098/rstb.2003.1354.

[6] E. Todorov, Emanuel & Erez, Tom & Tassa, Yuval. (2012). MuJoCo: A physics engine for model-based control. Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems. IEEE/RSJ International Conference on Intelligent Robots and Systems. 5026-5033. 10.1109/IROS.2012.6386109.

[7] J. Lofland, "Students' Case Studies of Social Movements: Experiences with an Undergraduate Seminar", Teaching Sociology, vol. 24, no. 4, p. 389, 1996. Available: 10.2307/1318877.

[8] E. Todorov, "Chapter 1: Overview," MuJoCo Overview, 2018. [Online]. Available: http://www.mujoco.org/book/index.html. [Accessed: 01-May-2020].